

ML2++ Tutorial for Evaluating IoT-Oriented Time-Series Workflows

Zahra Mardani Korani — LNEC & ISCTE-IUL / ISTAR, Lisbon, Portugal (zmardani@lnec.pt)

Supervisors: Alberto Rodrigues da Silva, Moharram Challenger, Armin Moin, João Carlos Ferreira

February 2026

1 Scenario Overview

ML2++ is an **MDE (model-driven engineering) tool** that uses a domain-specific modeling language to specify **IoT applications with integrated machine learning and time-series forecasting**. From a model, ML2++ generates IoT orchestration code and ML pipeline code such as preprocessing, training, prediction, and plots.

This tutorial uses **TejoFlow**, an **IoT river-monitoring use case** that forecasts **daily discharge** for the Tejo River using historical data from **1984–2023**. The goal is to show how ML2++ can support an end-to-end workflow in a single executable model: *data exchange* (IoT messages), *preprocessing*, *forecasting*, and *decision logic* (*safe/alert*) for this prediction scenario.

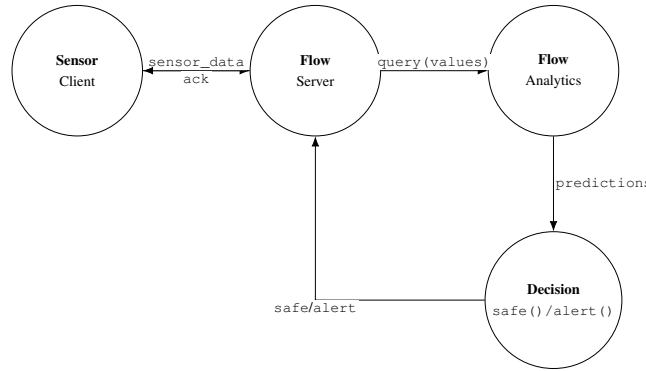


Figure 1: TejoFlow overview (data pipeline toward FlowAnalytics).

As indicated in Fig. 1, the system includes a decision algorithm and three ThingML components:

- **SensorClient**: gathers data and sends daily discharge values from the *Fratel*, *Castelo de Bode*, and *Almouroul* stations.
- **FlowServer**: forwards incoming values from sensor clients to **FlowAnalytics** and returns *ack*.
- **FlowAnalytics**: runs the ML pipeline (preprocess → train → predict) and triggers *safe()* or *alert()* based on a predefined threshold.

Reference tables. Appendix B includes: (i) Table 2 (ML2++ directives) and (ii) Table 3 (ThingML syntax used in this tutorial).

The full ML2++ model instance is provided in Appendix B (Listing 2), including the three components, message definitions, and the `data_analytics` configuration such as data, preprocessing, time-series setup, model, evaluation, and visualization settings.

The ML2++ model defines the messages in `FlowMsgs` and interconnects components: `SensorClient` → `FlowServer` → `FlowAnalytics`. `FlowAnalytics` contains the `data_analytics` block (`dal`) and executes the ML pipeline: it preprocesses and trains once at startup, then handles each incoming `query` by running prediction, saving results, and triggering *safe()* or *alert()* events.

2 Quick-Start Editor Tasks

This part is organized as a set of user **tasks** performed in the ML2++ Quick-Start Editor. Complete tasks in order. If something fails, take a screenshot of the Console Output and report it to the experimenter.

2.1 Opening the Editor and Logging In

Task 1. Open the editor

Open the ML2++ Quick-Start Editor using the URL provided by the instructor (e.g., shared by email or in the session handout): `http://ml2plusplus.jvmhost.net/`.

Task 2. Log in

Use **your own credentials** provided by the experimenter via email. Each participant receives a **different username and password** (do not share credentials with others).

Task 3. Identify workspace elements

After logging in, verify that the *Workspace* page contains:

- a **Model Editor** text area,
- action buttons: **Save Model**, **Show Preprocessing Plot**, **Generate Code**, **Download Code**, **Run Code**, and **Show Forecasting Plots**,
- a **Console Output** panel at the bottom of the page (logs from preprocessing, code generation, and execution).

2.2 Loading the TejoFlow Model

Task 4. Clear the editor

In the **Model Editor**, delete any existing content.

Task 5. Paste the sample model

Copy the complete model from Appendix B (Listing 2) *or* from the file attached in the email (model instance) and paste it into the **Model Editor**. From this point onward, all actions apply to this specific model instance.

Task 6. Save the model

Click **Save Model**. A confirmation message should appear.

2.3 Executing the Workflow

Task 7. Show the preprocessing plot

Click **Show Preprocessing Plot** to run the preprocessing script and visualize the resampled time series. Progress and log messages appear in **Console Output**.

Task 8. Generate code

Click **Generate Code** to generate executable artifacts, including: (i) ThingML-generated **Java** code for IoT orchestration and communication, and (ii) **Python** scripts for data analytics (preprocessing, training, prediction, and plotting).

Task 9. Download code

Click **Download Code** to download a `.zip` file with the generated Java project and Python scripts.

Tip. Where the generated code is located

After extracting (e.g., `Generated_user1.zip`), open `python_java/`:

- **Java:** `src/main/java/`
- **Python:** `src/python-scripts/` (`preprocess.py`, `train.py`, `predict.py`)

Tip. Run locally (optional)

`mvn clean install` then `java -jar target/<jarname>.jar`.

Dataset: `data/usecase1.csv`. Outputs: `data/`, `plots/`.

Task 10. Run code

Click **Run Code**. Inspect **Console Output** to follow preprocessing, training, prediction, and decision outputs (`safe()` / `alert()`).

Task 11. Show forecasting plots

Click **Show Forecasting Plots** to visualize forecasting outputs configured in the `model` block.

2.4 Exercise 1: Modifying Model Parameters

In this exercise, you modify forecasting parameters directly in the ML2++ model and observe how the generated pipeline changes after re-execution, without manually editing any Python code.

2.4.1 Locating the Model Block

The `model` block of `da1` is shown in the full model listing (Listing 2). For convenience, the relevant fragment is repeated in Listing 1.

```
model {
  autoML OFF
  algorithm MLP mymlp(
    hidden_layer_sizes (90,90),
    input_activation relu,
    hidden_activation relu,
    output_activation relu,
    regularization 12,
    dropout 0.2,
    optimizer adam,
    rate 0.001,
    batch_size 16,
    epochs 20,

    early_stopping disable,
    overfitting_Plots TRAINING_LOSS,
    forecasting_plots FORECAST_VS_ACTUAL
  )
  training_results "data/training.txt"
}
```

Listing 1: MLP model block in `da1`.

Parameter quick reference. Table 1 summarizes the meaning of the most relevant model parameters used in this exercise.

Parameter	Meaning / effect
<code>hidden_layer_sizes</code>	Neurons per hidden layer (e.g., (90,90) = two layers). Larger values may increase capacity and training time.
<code>dropout</code>	Dropout rate (e.g., 0.2). Higher values reduce overfitting but may increase underfitting.
<code>regularization</code>	Weight regularization type (e.g., l2).
<code>optimizer</code>	Optimization algorithm (e.g., adam).
<code>rate</code>	Learning rate. Too high may diverge; too low may train slowly.
<code>batch_size</code>	Samples per gradient update. Affects stability and speed.
<code>epochs</code>	Number of training epochs. More epochs increase time and may overfit.
<code>early_stopping</code>	Stops early if validation performance does not improve (if enabled).
<code>overfitting_Plots</code>	Training/validation diagnostic plots (e.g., loss curves).
<code>forecasting_plots</code>	Forecast plots (e.g., forecast vs actual).

Table 1: Quick reference for selected `model` parameters used in the tutorial.

Task 12. Modify parameters using auto-completion

Important. Keep attributes in the grammar-defined order; otherwise, the generator may map values to the wrong fields.

The Quick-Start Editor supports **auto-completion**:

- Windows/Linux: **Ctrl+Space**
- macOS: **Cmd+Space**

When you press **Ctrl+Space** (Windows/Linux) or **Cmd+Space** (macOS), it shows a list of valid keywords, options, or parameters that can appear right where your cursor is, and you can pick one to insert automatically.

Example: If your cursor is inside `training { ... }` and you type `epo`, then press **Ctrl+Space**, the editor may suggest `epochs`—select it to insert `epochs` automatically.

Update the `model` block by changing the forecasting algorithm and one training parameter. For example:

- replace `algorithm MLP ...` with `algorithm LSTM ...`,
- add `return_sequences true` immediately after the hidden-layer definition,
- reduce `epochs` (e.g., $20 \rightarrow 10$).

Re-run the workflow in the editor: **Save Model** → **Generate Code** → **Run Code** → **Show Forecasting Plots**. Observe how training time and the generated learning/forecasting curves change.

Task 13. Add an additional preprocessing plot

In the `visualization` block, add one additional preprocessing plot using auto-completion. `HEAT_MAP`

```
visualization {
  plots LINE_PLOT, HEAT_MAP
}
```

Re-run: **Save** → **Generate Code** → **Run Code**.

In the web editor, plots are shown via the plot buttons. When executing the downloaded project locally, figures are saved in a `plots/` (or similarly named) output folder.

Where to configure plots.

- **Preprocessing plots** are configured in the `visualization` block.
- **Overfitting and forecasting plots** are configured in the `model` block.

A Questionnaires

Instructions. After completing *all* tutorial tasks (Tasks 1–13), open the questionnaire link(s) below and submit your answers.

Main post-study questionnaire.

ML2++ Tutorial User Study — Post-Study Questionnaire

- Form A: Background & experience
- Form B: Task feedback (clarity, effort, confidence)
- Form C: SUS (System Usability Scale)
- Form D: NASA-TLX (Workload)

B Model Listing and Reference Tables

```
datatype String<256>
@type_checker " String "
@c_type " char *"
@java_type "String"
@js_type "String"

datatype Boolean<1>
@type_checker "Boolean"
@c_type "uint8_t"
@java_type "boolean"
@js_type "boolean"

datatype Int32<4>
@type_checker "Integer"
@c_type "int32_t"
@java_type "int"
@js_type "int"

thing fragment FlowMsgs {
  // Daily discharge values (m3/s): inputs = Fratel + Castelo de Bode, target = Almourol
  message sensor_data(fratel_discharge : Int32, castelo_bode_discharge : Int32, almourol_discharge : Int32)

  // ACK now includes a status string for tutorial consistency
  message ack(status : String)

  message query(fratel_discharge : Int32, castelo_bode_discharge : Int32, almourol_discharge : Int32)
  message alert()
  message safe()
}

thing SensorClient includes FlowMsgs {
  required port data_service {
    sends sensor_data
    receives ack
  }

  // Simulated discharge values (replace with real readings in a real deployment)
  property my_fratel_discharge : Int32 = 100
  property my_castelo_bode_discharge : Int32 = 120
  property my_almourol_discharge : Int32 = 110

  statechart SensorClientBehavior init SendData {

    state SendData {
      on entry do
        print "\n=====\\n"
        print "SensorClient started (DAILY cycle)\\n"
        print "In Quick-Start, the daily schedule is external.\\n"
        print "=====\\n"
```

```

    print "Sending discharge packet (Fratel, Castelo de Bode, Almourol)...\n"
    data_service!sensor_data(my_fratel_discharge, my_castelo_bode_discharge, my_almourol_discharge)
end

transition -> ReceiveAck
event a : data_service?ack
action do
    print "ACK received from server. Status = "
    print a.status
    print ". Packet delivered.\n"
end
}

state ReceiveAck {
    transition -> Stop
}

state Stop {
    on entry print "SensorClient finished.\n"
}
}

thing FlowServer includes FlowMsgs {
    provided port data_service {
        sends ack
        receives sensor_data
    }

    required port fa_service {
        sends query
        receives alert
        receives safe
    }
}

property fratel_discharge : Int32
property castelo_bode_discharge : Int32
property almourol_discharge : Int32

statechart FlowServerBehavior init GetData {

    state GetData {
        internal
        event e : data_service?sensor_data
        action do
            fratel_discharge = e.fratel_discharge
            castelo_bode_discharge = e.castelo_bode_discharge
            almourol_discharge = e.almourol_discharge
            print "Received packet. Forwarding to FlowAnalytics...\n"
            fa_service!query(fratel_discharge, castelo_bode_discharge, almourol_discharge)
        end

        transition -> Ack
        event fa_service?safe

        transition -> Alert
        event fa_service?alert
    }

    state Ack {
        on entry do
            print "SAFE. Sending ACK(SAFE) back to SensorClient.\n"
            data_service!ack("SAFE")
        end
        transition -> GetData
    }

    state Alert {
        on entry do
            print "ALERT. Sending ACK(ALERT) back to SensorClient.\n"
            data_service!ack("ALERT")
        end
        transition -> GetData
    }
}

```

```

    }
}

thing FlowAnalytics includes FlowMsgs {
  provided port fa_service {
    sends alert
    sends safe
    receives query
  }

  // Latest inputs from server
  property fratel_discharge : Int32
  property castelo_bode_discharge : Int32
  property almourol_discharge : Int32

  // 3-step prediction outputs (t+1, t+2, t+3)
  property predction1 : Int32
  property predction2 : Int32
  property predction3 : Int32

  data_analytics dal @dalib "keras-tensorflow" {

    data {
      dataset "usecase1.csv" // Hosted on the ML2++ platform: Tejo River daily discharge dataset (1984/2023)
      labels ON // CSV contains a header row (column names)

      features fratel_discharge, // Input feature 1: discharge at Albufeira do Fratel
               castelo_bode_discharge, // Input feature 2: discharge at Albufeira de Castelo de Bode
               almourol_discharge // Feature 3: discharge at Almourol (also used as the target below)

      output_features almourol_discharge // Prediction target (dependent variable): Almourol discharge

      timestamps ON // Dataset includes a timestamp column (used as time index for resampling/ordering)
      common_period_threshold 10 // Align multiple series; keep only periods where at least 10 common time points
      exist
    }

    preprocessing {
      resample DAILY // Force daily frequency (fill/align to one value per day before supervised conversion)
    }

    time_series {
      sequential TRUE // Treat data as sequences (required for time-series windowing)
      steps 3 // Forecast horizon: predict next 3 days (t+1, t+2, t+3)
      lag 20 // Input window size: use the past 20 days as model input
      multivariate ON // Use multiple input features jointly
      supervised_learning ON // Convert the time series into supervised (X,y) using sliding windows
    }

    model {
      autoML OFF
      algorithm MLP mlp_model(
        hidden_layer_sizes (90,90),
        input_activation relu,
        hidden_activation relu,
        output_activation relu,
        regularization 'l2',
        dropout 0.2,
        optimizer adam,
        rate 0.001,
        batch_size 32,
        epochs 20,
        early_stopping 'disable',
        overfitting_plots TRAINING_LOSS,
        forecasting_plots FORECAST_VS_ACTUAL
      )
      training_results "data/training.txt"
    }

    evaluation {
      prediction_results predction1, predction2, predction3
      metrics MSE
    }
  }
}

```

```

    }

    visualization {
        plots LINE_PLOT
    }
}

statechart FlowAnalyticsBehavior init Preprocess {

    state Preprocess {
        on entry do
            print "Preprocessing: resample DAILY + lag/steps + supervised conversion...\n"
            da_preprocess dal
        end
        transition -> Train
    }

    state Train {
        on entry do
            print "Training MLP model...\n"
            da_train dal
        end
        transition -> Ready
    }

    state Ready {
        on entry print "Ready for prediction requests.\n"
        transition -> Predict
        event m : fa_service?query
        action do
            fratel_discharge = m.fratel_discharge
            castelo_bode_discharge = m.castelo_bode_discharge
            almourol_discharge = m.almourol_discharge
        end
    }

    state Predict {
        on entry do
            print "Predicting next 3 days for Almourol...\n"
            da_predict dal (fratel_discharge, castelo_bode_discharge, almourol_discharge)
        end
        transition -> Decide
        on exit da_save dal
    }

    state Decide {
    on entry do
        print "Decision: checking flood threshold (DATA-DRIVEN, daily)...\n"

        if (prediction1 >= 2393 or prediction2 >= 2393 or prediction3 >= 2393)
            fa_service!alert()
        else
            fa_service!safe()
        end
        transition -> Ready
    }
}

configuration RiverFlowCfg @compiler "python_java" {
    instance sensorClient : SensorClient
    instance flowServer : FlowServer
    instance flowAnalytics : FlowAnalytics

    connector sensorClient.data_service => flowServer.data_service
    connector flowServer.fa_service => flowAnalytics.fa_service
}

```

Listing 2: Complete ML2++ model instance used in the tutorial.

Table 2: ML2++ Data Analytics Block – Detailed Description of Directives

Directive	Description
<i>Data section</i>	
dataset	Mandatory. Path to the time-series dataset file used for training and prediction.
labels	Optional. Indicates presence of a header/labels (ON, OFF, SEMI).
features	Optional. List of input features (independent variables).
input_features	Optional. Explicit list of model inputs (if different from <code>features</code>).
output_features	Optional. Target feature(s) to predict (dependent variables).
timestamps	Optional. Whether the dataset contains an explicit time index.
common_period_threshold	Optional. Threshold for aligning multiple time series to a common time period.
<i>Preprocessing section</i>	
preprocess_feature_scaler	Optional. Scaling strategy (e.g., StandardScaler, MinMaxScaler).
preprocess_sample_normalizer	Optional. Sample normalization strategy (e.g., L2, L1, max norm).
fill_missing	Optional. Missing-value handling (e.g., forward fill, interpolation, mean).
remove_outliers	Optional. Enables/disables outlier removal procedures.
advanced_imputation	Optional. Enables advanced/model-based imputation of missing data.
lag_features	Optional. Creates lagged versions of features as additional inputs.
rolling_window	Optional. Rolling-window statistics (e.g., moving average, rolling std).
resample	Optional. Resampling frequency (e.g., MINUTELY, HOURLY, DAILY).
transformations	Optional. Higher-level transformations (e.g., log transform, differencing, smoothing).
<i>Time-series and supervised-learning section</i>	
sequential	Optional. Treat input as sequential/time series.
steps	Optional. Number of future steps to predict.
lag	Optional. Past window size (number of lagged steps).
multivariate	Optional. Enable multivariate modeling with multiple inputs.
stationary_check	Optional. Enable stationarity tests and diagnostics.
seasonality_detection	Optional. Automatic seasonality detection (e.g., Fourier or moving-average based).
supervised_learning	Optional. Construct supervised dataset (X, y).
create_lagged_features	Optional. Explicit flag to generate lagged features in supervised dataset.
sliding_window	Optional. Sliding-window strategy for building input–output pairs.
<i>Model section</i>	
autoML	Optional. Enable/disable automated model/parameter selection.
algorithm	Mandatory. Algorithm to use (e.g., MLP, LSTM, ARIMA).
training_results	Optional. File path where training logs/metrics are stored (e.g., "data/training.txt").
hyperparameter_tuning	Optional. Hyperparameter search strategy (grid/random/Bayesian).
ensemble_methods	Optional. Ensemble method (bagging/boosting/stacking).
<i>Evaluation and analysis section</i>	
prediction_results	Optional. Model properties storing prediction outputs (e.g., 3-step forecasts).
metrics	Optional. Metrics (e.g., RMSE, MAE, MSE, R_SQUARED).
outlier_detection	Optional. Post-hoc outlier detection on predictions/residuals.
clustering	Optional. Time-series clustering/grouping analysis.
context	Optional. Application context declaration.
<i>Visualization section</i>	
plots	Optional. List of plots to generate (preprocessing and/or forecasting plots).
Important. Keep attributes in the DSL (grammar-defined) order; otherwise, the generator may map values to the wrong fields.	

Table 3: ThingML Syntax Commands

Command/Keyword	Purpose	Example
thing	Declare a Thing/component	thing SensorClient { ... }
thing fragment	Reusable message fragment	thing fragment FlowMsgs { ... }
message	Define message	message sensor_data(...)
provided port	Declare provided port	provided port fa_service { ... }
required port	Declare required port	required port data_service { ... }
property	Internal variable	property x : Int32 = 10
statechart	Define behavior	statechart Behavior init S { ... }
state	Declare state	state Predict { on entry do ... end }
on entry do ... end	Actions on entering a state	on entry do da_train dal end
transition → State	Transition to another state	transition -> Ready
transition ... event port?msg	Transition on message event	transition -> Predict event fa_service?query
internal event e: port?msg	Capture incoming message	internal event e : data_service?sensor_data
action do ... end	Execute multiple actions	action do x = e.value end
if (cond) ... else ...	Conditional execution	if (p1 >= 150) fa_service!alert() else fa_service!safe()
stop	Stop state machine	stop
connector	Connect ports	connector A.p => B.p
instance	Instantiate a Thing	instance flowServer : FlowServer
da_preprocess	Trigger preprocessing	on entry do da_preprocess dal end
da_train	Trigger training	on entry do da_train dal end
da_predict	Trigger prediction	on entry do da_predict dal(...) end
da_save	Save model state	on exit do da_save dal end